

Independent Verification Mitigates Over-Validation in Multi-Agent LLM Tutoring Systems

Oussama Chakroun
Computer Science, ISIMM
Tunisia
chakroun.oussama003@gmail.com

Isra Moussaoui
Computer Science, ISIMM
Tunisia
isramoussaoui01@gmail.com

Abstract—Large language models (LLMs) are increasingly used as automated tutors that grade a student’s next proof step and issue a verdict (correct, suboptimal, or incorrect). A single LLM tutor grading blind, without ground truth, is prone to *over-validation*: accepting an incorrect step as correct. We study whether adding an independent second grader (a Verifier) and a tie-breaking third grader (Recovery), invoked only when the Tutor and Verifier disagree, reduces over-validation relative to a single-agent baseline. We evaluate this three-role pipeline on a dataset of propositional-logic proof steps, comparing single-agent Tutor verdicts against the full pipeline’s final verdict on *matched* sets of proof states to eliminate sampling confounds, and testing significance with McNemar’s exact test. Across two independently implemented orchestrations, the pipeline reduces over-validation by 7–11 percentage points (35.8%→24.7%, $p=0.0001$, $n=503$; and 33.9%→26.9%, $p<0.0001$, $n=513$), while over-rejection shows no significant change in either run. This improvement comes at a measured $2.09\times$ latency cost (19.1s → 39.9s mean per state) relative to a Tutor-only call. We analyze the mechanism behind the improvement: Tutor and Verifier agree on $\sim 90\%$ of steps (correlated blind spots), but on the $\sim 10\%$ where they disagree, the tie-breaking Recovery role resolves the disagreement correctly in $\sim 86\text{--}87\%$ of cases, and disproportionately in the direction of catching over-validation rather than introducing new errors (a $\sim 10\text{:}1$ ratio of corrected-to-introduced errors in one run). We discuss the resulting accuracy-versus-cost tradeoff and its implications for deploying multi-agent grading in tutoring systems.

Index Terms—large language models, multi-agent systems, intelligent tutoring systems, automated grading, LLM-as-a-judge, over-validation, proof verification

I. INTRODUCTION

LLM-based tutoring systems promise to scale personalized, step-level feedback for students working through multi-step problems such as formal proofs [1]. A core component of such systems is a *grading* step: given a student’s proposed next step, the tutor must decide whether it is correct, an acceptable alternative, or incorrect, typically without access to ground truth at inference time (a “blind” grading setting, analogous to LLM-as-a-judge evaluation [2]).

A single LLM tutor grading blind is subject to a specific and pedagogically costly failure mode: *over-validation*, in which the tutor accepts a genuinely incorrect step as correct. This is closely related to sycophancy and self-consistency biases documented in LLM evaluators more broadly [3], and to the finding that LLM tutors, even when capable of solving the underlying problem, struggle to precisely detect and localize

student errors [1]. In a tutoring context, over-validation is more harmful than the converse error (over-rejection, incorrectly flagging a valid step as wrong): a student who is told an error is correct will build on it, compounding the mistake, whereas a student who is incorrectly told a correct step is wrong will typically re-derive it and recover.

Multi-agent approaches – debate [4], self-refinement [5], and chain-of-verification [6] – have been proposed to reduce single-model error rates by introducing a second pass of scrutiny. We adapt this idea to the tutoring-grading setting with a specific architectural choice: rather than debate or self-critique (where a model sees and can be anchored by its own or another’s prior output), we use a **blind, independent second grader** (Verifier) that receives the same inputs as the Tutor but never sees the Tutor’s verdict, followed by a **tie-breaking third grader** (Recovery) invoked only when the first two disagree. This design is intended to avoid the correlated-error risk of debate protocols, where two instances of a similar model can converge on the same mistake once one has seen the other’s reasoning.

Our contributions are:

- We show, on a matched, paired sample of proof states, that this Tutor–Verifier–Recovery pipeline produces a statistically significant reduction in over-validation relative to a single-agent Tutor baseline, using McNemar’s exact test to control for the paired, non-independent nature of the comparison.
- We replicate this finding across two independently implemented orchestrations (a linear script-based pipeline and a graph-based orchestration built with LangGraph), drawn from a shared pool of underlying proof states, strengthening the claim that the effect is robust to orchestration/implementation choices rather than an artifact of one particular pipeline’s code.
- We report, for the first time in this setting to our knowledge, the *latency cost* of the added verification stages, showing a $2.09\times$ overhead relative to a Tutor-only call, and we decompose this cost by pipeline stage.
- We analyze the mechanism: high Tutor–Verifier conformity ($\sim 90\%$) indicates shared blind spots, but the tie-breaking Recovery role, when invoked, resolves disagreements correctly in the large majority of cases and disproportionately corrects over-validation errors rather

than introducing new ones.

II. RELATED WORK

LLM-as-a-judge. Using an LLM to grade or evaluate another model’s (or a human’s) output has become a standard technique for scalable evaluation [2], but is known to suffer from systematic biases, including a tendency toward agreement over correctness [3]. Our over-validation metric can be viewed as a task-specific instance of this general failure mode, measured against ground-truth proof validity rather than human preference.

Multi-agent verification and debate. Multi-agent debate has been shown to improve factuality and reasoning by having multiple LLM instances critique and refine answers over several rounds [4]. Self-Refine [5] and Chain-of-Verification [6] instead use a single model to critique or verify its own draft output through a structured second pass. Our architecture differs from both: the Verifier is never conditioned on the Tutor’s output (unlike self-refinement, and unlike debate after the first round), which is intended to preserve independence between the two verdicts rather than letting the second grader anchor on the first.

Multi-agent failure modes. Systematic analysis of multi-agent LLM systems has identified 14 distinct failure modes across three categories: specification and design issues, inter-agent misalignment, and task verification failures [7]. Critically, accumulating good agents does not guarantee a good system – when agents agree, they may be converging on a shared error rather than independently confirming a correct answer, a phenomenon documented as conformity bias. Our `conformity_rate` metric directly operationalizes this failure mode in the tutoring grading domain, and our Recovery role is a targeted architectural response to the verification failure category.

Prior mitigation attempts and their limitations. Several recent works have examined specifically why naive multi-agent aggregation fails to fix the correlated-error problem we observe, rather than just documenting that it occurs. One recent theoretical result disentangles multi-agent debate into its voting and deliberation components and shows that, when agents receive identical inputs, debate alone induces a martingale over agents’ belief trajectories: expected correctness does not improve across rounds, because later rounds add no genuinely new information, only restatements of correlated views; the bulk of any observed gain instead comes from majority voting itself, not from the exchange of arguments [8]. This formalizes, for debate generally, the same correlated-blind-spot mechanism we observe empirically as $\sim 90\%$ Tutor-Verifier conformity. A related empirical study finds that even a 9-judge, 7-model-family panel carries the effective statistical weight of only ~ 2 independent judges, and that even with oracle-calibrated aggregation methods (e.g., Dawid-Skene, accuracy-weighted voting), correlated judge errors leave most of the theoretical gap to an ideal independent-voting aggregator unclosed – indicating the bottleneck is the correlation between judges’ errors, not the aggregation rule used to combine them

[9]. This supports our design choice to treat conformity as a property to measure and route around (via a re-derivation step) rather than something a smarter voting scheme could fix.

A second line of work has tested using an LLM itself as the tie-breaking adjudicator and found this can perform *worse* than doing nothing. One recent debate framework used an LLM-as-Judge baseline to decide when to overturn a majority vote and reported negative net gains from it, because the same correlated errors producing the initial disagreement also biased the adjudicator; the authors instead fell back to a lightweight non-LLM classifier trained on debate-log features to decide when to overturn a vote, achieving positive net gains only once the adjudication step was taken out of the LLM’s hands [10]. This is a useful counter-example to architectures like ours: it suggests that a third LLM call does not automatically help, and that our Recovery role’s effectiveness likely depends on its specific instruction to independently re-derive the proof step from the givens, rather than to weigh or vote between the two prior verdicts.

Finally, on the tutoring side specifically, prior work has established the error pattern but, to our knowledge, has not tested an architectural remedy for it. [11] and [12] document pedagogical sycophancy and over-validation as persistent, model-general failure modes but limit their contribution to identifying and measuring the problem. Separately, an evaluation of LLM tutors across several step-based tutoring platforms finds that models perform at or below chance when identifying genuinely incorrect student actions, despite being comparatively reliable at confirming correct ones [13] – reinforcing that the over-validation asymmetry we target is not an artifact of our particular benchmark or model, but a broader pattern that no prior tutoring-specific work appears to have addressed with an independent-verification architecture.

LLM tutoring and error detection. Prior work has found that LLM tutors, even when capable of solving the underlying problem, struggle to precisely detect and localize student errors, and that adding an explicit verification step before feedback generation improves response quality [1]. Pedagogical sycophancy – where an LLM tutor validates incorrect student answers rather than correcting them – has been identified as an educational safety risk [11], distinct from general LLM sycophancy in that it occurs in a high-trust, low-verification context where students are unlikely to independently check the tutor’s judgment. Most directly related to our setting, Yasir et al. [12] benchmark seven LLM feedback agents on propositional-logic proof steps and find systematic over-validation and over-rejection failures that persist across models, attributing them to architectural rather than informational limits. Our work proposes and evaluates an architectural intervention – independent multi-agent verification – intended to address exactly the failure modes they document.

III. SYSTEM ARCHITECTURE

A. Roles

The pipeline consists of four roles, each backed by an LLM call:

- **Student:** proposes a candidate next step (a logical inference, its justifying rule, and parent statements) given the problem’s givens, intermediate steps, and conclusion. The Student role is used only to generate candidate steps for evaluation; at inference time in a real tutoring system, this role would be played by a human student.
- **Tutor:** grades the Student’s step blind – given only the problem context and the submitted step, with no access to ground truth – and returns a verdict (Correct / Suboptimal / Incorrect) with feedback.
- **Verifier:** performs the same grading task independently. By design, the Verifier prompt never includes the Tutor’s verdict or feedback; it receives exactly the same inputs the Tutor received (problem context plus the submitted step) and nothing more.
- **Recovery:** a tie-breaker invoked only when the Tutor’s and Verifier’s verdicts disagree. It receives the full problem context, the submitted step, and *both* graders’ verdicts and feedback (labeled “Grader A” and “Grader B”), and is explicitly instructed to independently re-derive what steps are validly derivable from the givens, rather than deferring to either grader or averaging their verdicts. It outputs a final verdict and a brief justification of how the disagreement was resolved.

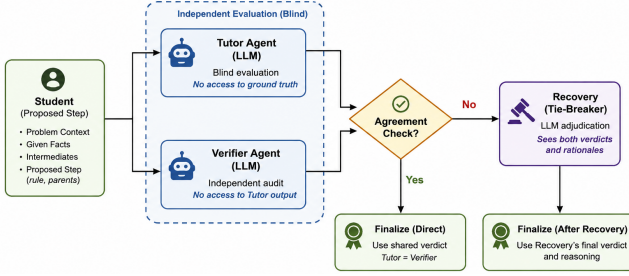


Fig. 1. Multi-agent tutoring architecture with independent Tutor and Verifier evaluation, followed by agreement-based adjudication and final decision.

When Tutor and Verifier agree, their shared verdict becomes the pipeline’s final verdict directly; Recovery is skipped. This conditional invocation is what produces the latency/cost asymmetry analyzed in Section V.

B. Design rationale

The central design choice is grader *independence*: the Verifier is blind not only to ground truth but to the Tutor’s opinion. This is intended to avoid the failure mode in which two similar models, once one has seen the other’s verdict, converge on a shared error rather than catching it – a risk documented in debate-style architectures where later rounds are conditioned on earlier ones [4]. Only the Recovery role, which exists specifically to resolve disagreement, is given both prior verdicts, and it is explicitly prompted not to default to either one.

C. Implementations

We implemented and evaluated this architecture in two independent codebases: (1) a linear, script-based orchestration, and (2) a graph-based orchestration built with LangGraph (Tutor → Verifier → Compare → Recovery (if needed) → Finalize). Both experiments draw their evaluation states from the same underlying pool of proof states; as such, the two experiments should be read as a robustness check across orchestration implementations rather than as replication on independent samples. We report both results separately rather than pooling them for this reason. Moreover, every role (Student, Tutor, Verifier, Recovery) is served by the identical model, Qwen2.5-7B-Instruct, run locally with no external API dependency, rather than assigning different models to different roles. We treat this as a methodological control for model-capability confounds: any improvement is attributable to the architecture rather than a stronger judge model. Note, however, that this also means the independence between Tutor and Verifier is prompt-level independence within a single model’s distribution, not independence across genuinely different models, a point we return to in Section VII given the high conformity rate we observe.

IV. DATASET AND EXPERIMENTAL SETUP

A. Data

Proof states are drawn from the BEA 2026 propositional-logic tutoring benchmark [12], a corpus of 10,836 proof steps paired with ground-truth labels derived from a knowledge graph of valid inference paths. Each proof state specifies the problem’s givens, intermediate steps derived so far, and target conclusion; the ground-truth label reflects the step actually submitted by the Student model: *optimal* (the single best valid next step), *valid_alternative* (a logically valid but non-optimal step), or *incorrect*. States are drawn via stratified sampling by problem, with a fixed random seed ($seed=42$) shared across all runs to keep the underlying case mix comparable across experiments. After excluding states where either the baseline or pipeline run returned a parse failure, the matched evaluation subset contains $N = 503$ proof states. The rule vocabulary spans standard propositional-logic inference and equivalence rules (Modus Ponens, Modus Tollens, Conjunction, Addition, Disjunctive Syllogism, Hypothetical Syllogism, De Morgan’s Laws, and others). The dataset is released under IRB approval with informed consent. We use the dataset in accordance with the conditions specified by the dataset authors.

B. Metrics

Let a proof state’s ground-truth label be $g \in \{\text{optimal}, \text{valid_alternative}, \text{incorrect}\}$ and a system’s verdict be $v \in \{\text{Correct}, \text{Suboptimal}, \text{Incorrect}\}$, with the natural mapping $\text{Correct} \rightarrow \text{optimal}$, $\text{Suboptimal} \rightarrow \text{valid_alternative}$, $\text{Incorrect} \rightarrow \text{incorrect}$. We report:

- **over_validation_rate:** among states with $g=\text{incorrect}$, the fraction graded $v \neq \text{Incorrect}$. *This is the paper’s primary outcome.*

- **over_rejection_rate**: among states with $g \neq \text{incorrect}$, the fraction graded $v = \text{Incorrect}$.
- **exact_agreement_rate**: the fraction of all states where the mapped verdict exactly matches g .
- **conformity_rate** (pipeline only): the fraction of states where Tutor and Verifier verdicts agree with each other (regardless of g).
- **recovery_rate**: the fraction of states on which Recovery was invoked (i.e. Tutor and Verifier disagreed); by construction, $\text{recovery_rate} = 1 - \text{conformity_rate}$.
- **recovery_precision**: among states where Recovery was invoked, the fraction where Recovery’s final verdict matched g .

C. Matched-sample comparison and significance testing

Because the Tutor’s verdict inside the full pipeline is computed independently of the Verifier and Recovery (it is graded before either runs), it is directly usable as the single-agent baseline verdict for the *same* proof state – avoiding the need for a separately-run baseline script and the sample-mismatch risk that entails. Where a separate baseline run was used (Experiment 1), we restrict both runs’ outputs to the intersection of proof-state IDs that were successfully parsed in both, ensuring the baseline and pipeline comparison is computed over an identical set of states. In both experiments we then compare the baseline (Tutor-alone) and pipeline (final) verdicts on the exact same paired states using McNemar’s exact test on the resulting 2×2 discordant-pair table, which is the appropriate test for paired binary outcomes on the same items, as opposed to treating the two systems’ error rates as independent samples.

V. RESULTS

We report three tables, each answering one question: Table I – *did the pipeline significantly reduce over-validation?*; Table II – *why, mechanically, does it work?*; Table III – *what does it cost?* Secondary checks (which grader Recovery tends to trust, and whether Recovery’s extra LLM call beats a free heuristic) are reported as single numbers in the Discussion rather than as separate tables, since each reduces to one comparison.

A. Accuracy

In both experiments, the pipeline produces a large, statistically significant reduction in over-validation relative to the Tutor-alone baseline (7–11 percentage points, $p < 0.001$), while over-rejection shows no significant change.

B. Mechanism

Tutor and Verifier agree on $\sim 90\%$ of states (conformity_rate), meaning most of their errors are shared rather than independent. On the $\sim 10\%$ where they disagree, Recovery – prompted to independently re-derive the step rather than defer to either grader – resolves the disagreement correctly $\sim 86\%$ of the time ($\text{recovery_precision}$), and does so asymmetrically: in Experiment 2, of the 44 discordant baseline/pipeline outcomes,

TABLE I
ACCURACY COMPARISON: SINGLE-AGENT TUTOR BASELINE VS. FULL PIPELINE (TUTOR+VERIFIER+RECOVERY), ON MATCHED, PAIRED PROOF STATES. MCNEMAR’S EXACT TEST ON PAIRED, DISCORDANT OUTCOMES.

	Exp. 1 (script pipeline)	Exp. 2 (LangGraph)
Matched n	503	513
<i>over_validation_rate</i> (primary outcome)		
Baseline (Tutor alone)	35.79%	33.92%
Pipeline (final verdict)	24.65%	26.90%
Absolute reduction	−11.14 pp	−7.02 pp
McNemar p -value	0.0001	<0.0001
<i>exact_agreement_rate</i>		
Baseline $\rightarrow$ Pipeline	61.23% $\rightarrow$ 73.16%	62.77% $\rightarrow$ 69.79%
<i>over_rejection_rate</i> (not significant)		
Baseline $\rightarrow$ Pipeline	1.99% $\rightarrow$ 1.59%	2.53% $\rightarrow$ 2.53%
McNemar p -value	0.8036	1.0000

the pipeline corrected 40 over-validations and introduced only 4 new errors (a 10:1 ratio). Table II summarizes these three numbers, which together explain the accuracy gain in Table I.

TABLE II
PIPELINE MECHANISM, BOTH EXPERIMENTS.

	Exp. 1	Exp. 2
conformity_rate (Tutor/Verifier agree)	90.26%	90.06%
$\text{recovery_precision}$ (Recovery correct)	85.7%	86.27%
Corrected : introduced error ratio	–	10:1 (40:4)

C. Latency and cost

TABLE III
PER-STATE LATENCY, EXPERIMENT 2 (LANGGRAPH, $n = 513$).

Quantity	Value
Mean Tutor-only latency	19.13 s
Mean full-pipeline latency	39.91 s
Mean overhead ratio	$2.09 \times$

The pipeline takes $2.09 \times$ as long per state as a Tutor-only call. Because Recovery fires on only $\sim 10\%$ of states, most of this overhead comes from the Verifier stage, which runs on every state regardless of outcome.

D. Secondary checks: is the extra Recovery call worth it?

On the Experiment 1 disagreement subset ($n = 49$), Recovery’s actual accuracy (85.7%) beats both a zero-cost “always trust Tutor” heuristic (18.4%) and an “always trust Verifier” heuristic (77.6%) – but only by 8.2 points over the better heuristic, and Recovery in fact sides with the Verifier in 81.6% of these cases. Because disagreements are only $\sim 10\%$ of all states, this translates to a modest net gain in overall $\text{exact_agreement_rate}$ (+0.78 pp: 73.33% with Recovery vs. 72.55% under a Verifier-only fallback). In short, most of the architecture’s benefit comes from simply having an independent second grader; Recovery’s own LLM call adds a real but secondary increment on top of that.

VI. DISCUSSION

A. Why over-validation improves but over-rejection does not

The absence of a significant change in over-rejection is consistent with two non-exclusive explanations. First, the ground-truth label distribution is heavily skewed toward `incorrect` steps in this dataset (95.2% of the 503 matched states, versus 3.2% `optimal` and 1.6% `valid_alternative`), so the over-rejection metric is computed over a much smaller subset of states ($n = 24$ non-incorrect steps in total), giving the paired significance test comparatively little power in both experiments (only 16 and 2 discordant pairs, respectively, versus 192 and 44 for over-validation). Second, over-rejection may simply be a rarer and less systematically biased error for a single LLM tutor than over-validation, consistent with sycophancy-style biases pushing graders toward acceptance rather than rejection [3]; if the baseline error rate on this axis is already low, there is less room for a verification stage to improve it, and correspondingly less signal for a significance test at this sample size to detect.

B. Mechanism: correlated blind spots, corrected by tie-breaking

Tutor–Verifier conformity is high in both experiments ($\sim 90\%$), meaning the two independent graders agree on the large majority of steps. This indicates that Tutor and Verifier share correlated blind spots – plausibly because both are grading the same task with similar prompting – rather than making independent, uncorrelated errors that a simple majority-style aggregation would be expected to catch. The pipeline’s accuracy gain is concentrated almost entirely in the $\sim 10\%$ of states where the two graders disagree: this is precisely the situation Recovery is invoked for, and its high resolution accuracy combined with the 10:1 asymmetric correction ratio observed in Experiment 2 indicates that the tie-breaking step is doing genuine independent verification work rather than arbitrating between two opinions by default – though its tendency to side with the Verifier four times out of five (Section V.D) suggests it functions somewhat more like a Verifier-leaning tie-break than a fully symmetric third opinion.

C. The accuracy-cost tradeoff

The pipeline roughly doubles per-state latency – and, correspondingly, roughly doubles-to-quadruples inference cost relative to a single Tutor call, since Verifier always runs and Recovery runs on $\sim 10\%$ of states – in exchange for a 7–11 percentage point reduction in over-validation. Whether this tradeoff is worthwhile depends on the deployment context: in a synchronous, single-student tutoring session, a ~ 20 second added latency per graded step is a real user-facing cost; in an asynchronous or batch grading context (e.g., generating feedback on a submitted problem set), the added latency is far less consequential relative to the accuracy gain. We see this cost-accuracy characterization, rather than accuracy alone, as the more complete and more useful result for practitioners deciding whether to deploy multi-agent grading.

VII. LIMITATIONS

The interpretation of these findings should take into account several factors. First, the over-rejection comparison has limited statistical power in both experiments due to the class imbalance in the ground-truth label distribution. Therefore, the absence of a statistically significant difference should be interpreted cautiously rather than as evidence of no effect. Second, the $\sim 90\%$ Tutor–Verifier conformity indicates a high level of agreement between the two graders, while also suggesting that some systematic patterns may be shared between them, potentially due to similarities in prompting or training data. Introducing a more diverse third grading strategy could help further assess the extent to which such shared patterns influence the results. Finally, both experiments use samples drawn from a pool of 516 unique proof states; after excluding parse failures in either run, the matched evaluation subset contains $N = 503$ states. Evaluating the approach on additional proof-state pools would provide a broader empirical basis and further narrow the confidence intervals, particularly for the over-rejection comparison.

VIII. CONCLUSION

We presented and evaluated a Tutor–Verifier–Recovery multi-agent architecture for grading student proof steps in an LLM tutoring setting, in which a Verifier grades independently and blind to the Tutor’s opinion, and a Recovery role tie-breaks disagreements by re-deriving the proof step rather than arbitrating between the two prior verdicts. Across two independent implementations, this architecture produces a statistically significant reduction in over-validation – the more pedagogically costly of the two grading error types – at a measured, quantifiable latency cost of roughly $2\times$ a single-agent call. We view the combination of (a) a matched, paired significance test rather than raw rate comparison, (b) replication across independent implementations, and (c) an explicit accuracy-cost characterization as a more complete evaluation methodology for multi-agent LLM tutoring systems than accuracy improvement alone.

REFERENCES

- [1] N. Daheim, J. Macina, M. Kapur, I. Gurevych, and M. Sachan, “Stepwise verification and remediation of student reasoning errors with large language model tutors,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024, arXiv:2407.09136.
- [2] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging LLM-as-a-judge with MT-bench and chatbot arena,” in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023, arXiv:2306.05685.
- [3] M. Sharma, M. Tong, T. Korbak, D. Duvenaud, A. Askell, S. R. Bowman, N. Cheng, E. Durmus, Z. Hatfield-Dodds, S. R. Johnston, S. Kravec, T. Maxwell, S. McCandlish, K. Ndousse, O. Rausch, N. Schiefer, D. Yan, M. Zhang, and E. Perez, “Towards understanding sycophancy in language models,” *arXiv preprint arXiv:2310.13548*, 2023.
- [4] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, “Improving factuality and reasoning in language models through multiagent debate,” in *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024, arXiv:2305.14325.

- [5] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang *et al.*, “Self-refine: Iterative refinement with self-feedback,” *arXiv preprint arXiv:2303.17651*, 2023.
- [6] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, “Chain-of-verification reduces hallucination in large language models,” in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3563–3578, arXiv:2309.11495.
- [7] M. Cemri *et al.*, “Why do multi-agent LLM systems fail?” 2025.
- [8] H. K. Choi, X. Zhu, and Y. Li, “Debate or vote: Which yields better decisions in multi-agent large language models?” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025, arXiv:2508.17536.
- [9] G. Kohli, “Nine judges, two effective votes: Correlated errors undermine LLM evaluation panels,” 2026.
- [10] C. He, Z. Chen, Z. Yang, S. Qiao, M. Ju, J. Liu, D. Wen, and G. Liu, “Minority sentinel: When to overturn majority voting in multi-agent LLM debates,” in *AgentSearch Workshop @ SIGIR*, 2026, arXiv:2606.29270.
- [11] X. Theimer-Lienhard *et al.*, “Sycophancy is an educational safety risk: Why LLM tutors need sycophancy benchmarks,” 2026.
- [12] T. Yasir, W. Li, S. Gilson, S. D. Tithi, X. Tian, and T. Barnes, “Confirming correct, missing the rest: Llm tutoring agents struggle where feedback matters most,” in *Proceedings of the 21st Workshop on Innovative Use of NLP for Building Educational Applications (BEA)*, 2026.
- [13] D. Weitekamp, M. N. Siddiqui, and C. J. MacLellan, “TutorGym: A testbed for evaluating AI agents as tutors and students,” in *Proceedings of the 26th International Conference on Artificial Intelligence in Education (AIED)*, 2025, arXiv:2505.01563.